

Travail d'initiative personnelle encadré

Utilisation des machines à vecteurs de support dans la détection de visages par ordinateur

Jules DUPONT - Numéro de candidat: **18573**

Introduction



Figures: Illustration. Caméras de vidéosurveillance, Paris.

Reconnaissance faciale

- Authentifier une personne (1:1)
- Identifier une personne (1:N)

Étape préliminaire:

Détection de visages



Figure: Détection des visages présents sur une image

Introduction

- Réseaux de neurones convolutifs (CNN)
- **Histogrammes de gradient orienté + Machines à vecteurs de support (HOG + SVM)**

Introduction

- Réseaux de neurones convolutifs (CNN)
- **Histogrammes de gradient orienté + Machines à vecteurs de support (HOG + SVM)**

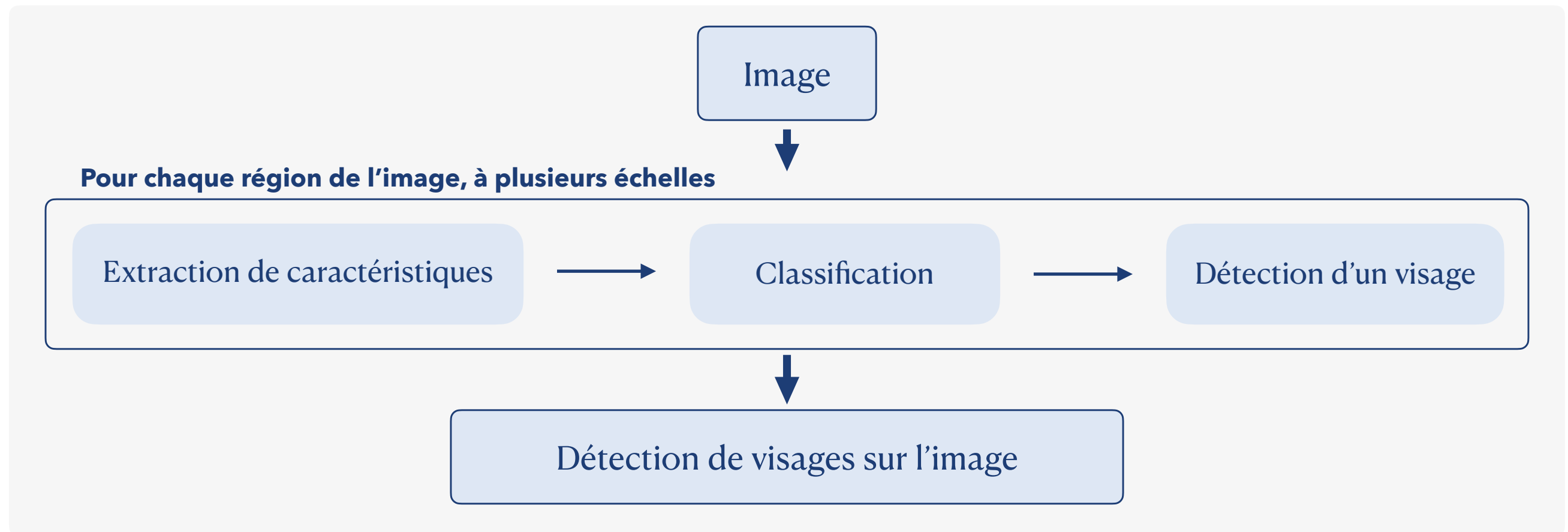


Figure: Processus de détection de visages avec un algorithme de type HOG+SVM

Introduction

- Réseaux de neurones convolutifs (CNN)
- **Histogrammes de gradient orienté + Machines à vecteurs de support (HOG + SVM)**

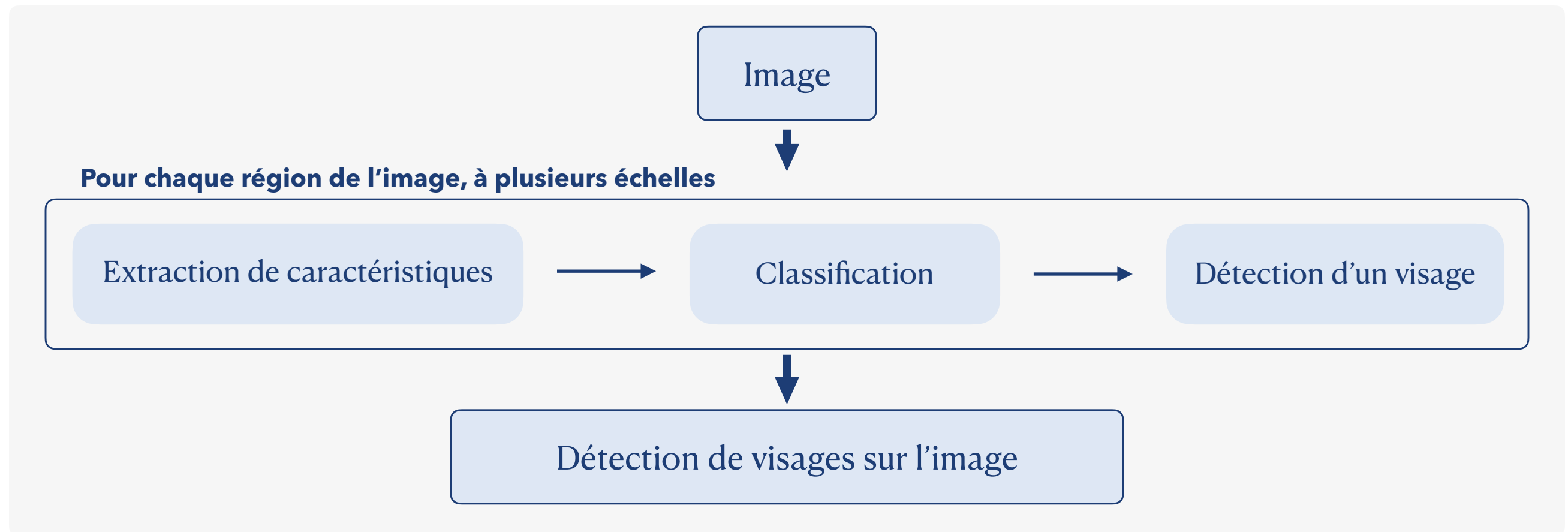


Figure: Processus de détection de visages avec un algorithme de type HOG+SVM

Problématique

En quoi l'utilisation de machines à vecteurs de support reste-t-elle pertinente dans la détection de visages par ordinateur ?

Sommaire

I. Extraction de caractéristiques

II. Classification

III. Mise en place de l'algorithme

I. Extraction de caractéristiques

I. Extraction de caractéristiques

1) Intérêt

Données visuelles brutes

```
[[0.90980392 0.90980392 0.90980392 ... 0.90980392 0.90980392 0.90980392]
 [0.90980392 0.90980392 0.90980392 ... 0.90980392 0.90980392 0.90980392]
 [0.90980392 0.90980392 0.90980392 ... 0.90980392 0.90980392 0.90980392]
 ...
 [0.93333333 0.93333333 0.93246676 ... 0.38943784 0.38422834 0.37131775]
 [0.93333333 0.93333333 0.93076157 ... 0.38294925 0.37259071 0.37000536]
 [0.93333333 0.93333333 0.93076157 ... 0.37480729 0.36746139 0.37553839]]
```

Figure: Avec Python, la bibliothèque OpenCV représente une image en niveau de gris par un tableau NumPy qui donne le niveau de gris de chaque pixel.



Extraction de
caractéristiques
(ou *features extraction*)

Caractéristiques visuelles

(ou *visual features*)

I. Extraction de caractéristiques

2) Histogrammes de gradient orienté (HOG)

[1] N. DALAL ET B. TRIGGS « Histograms of oriented gradients for human detection »

- décrire l'apparence et la forme locale d'un objet dans une image par la distribution de l'intensité du gradient

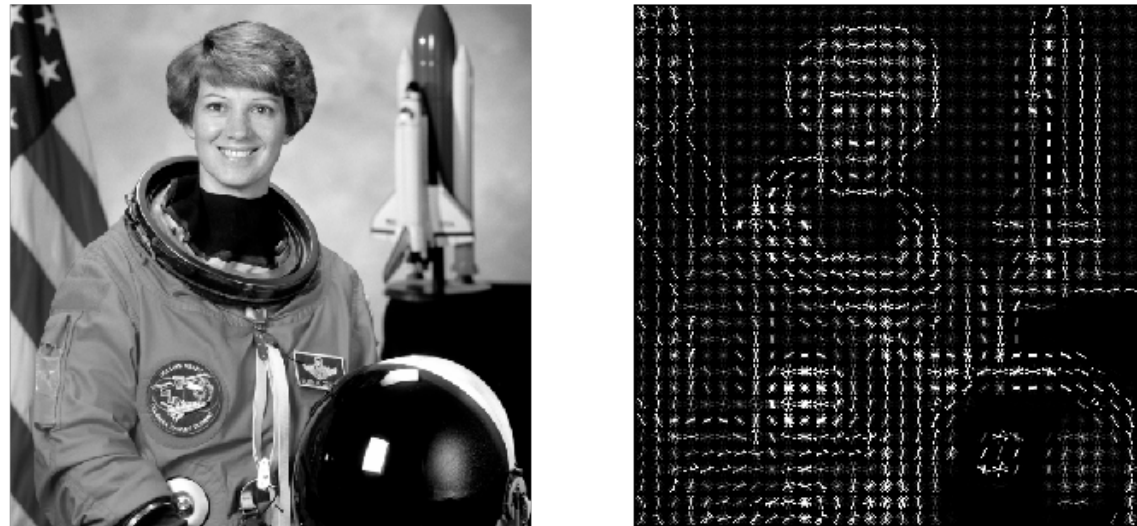


Figure 1: Visualisation des caractéristiques HOG d'une image

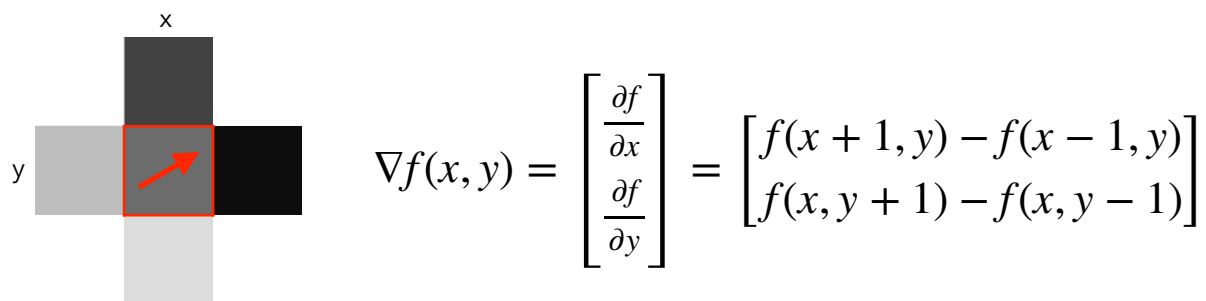


Figure 2: Calcul du gradient en un pixel de coordonnées (x, y)

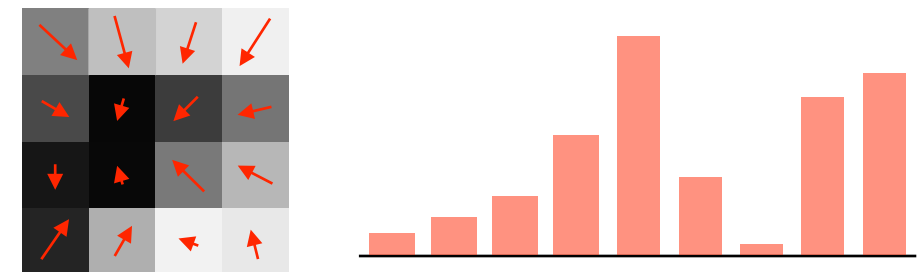


Figure 3: Calcul de l'histogramme de l'orientation des gradients en l'une des cellules

II. Classification

II. Classification

1) Intérêt

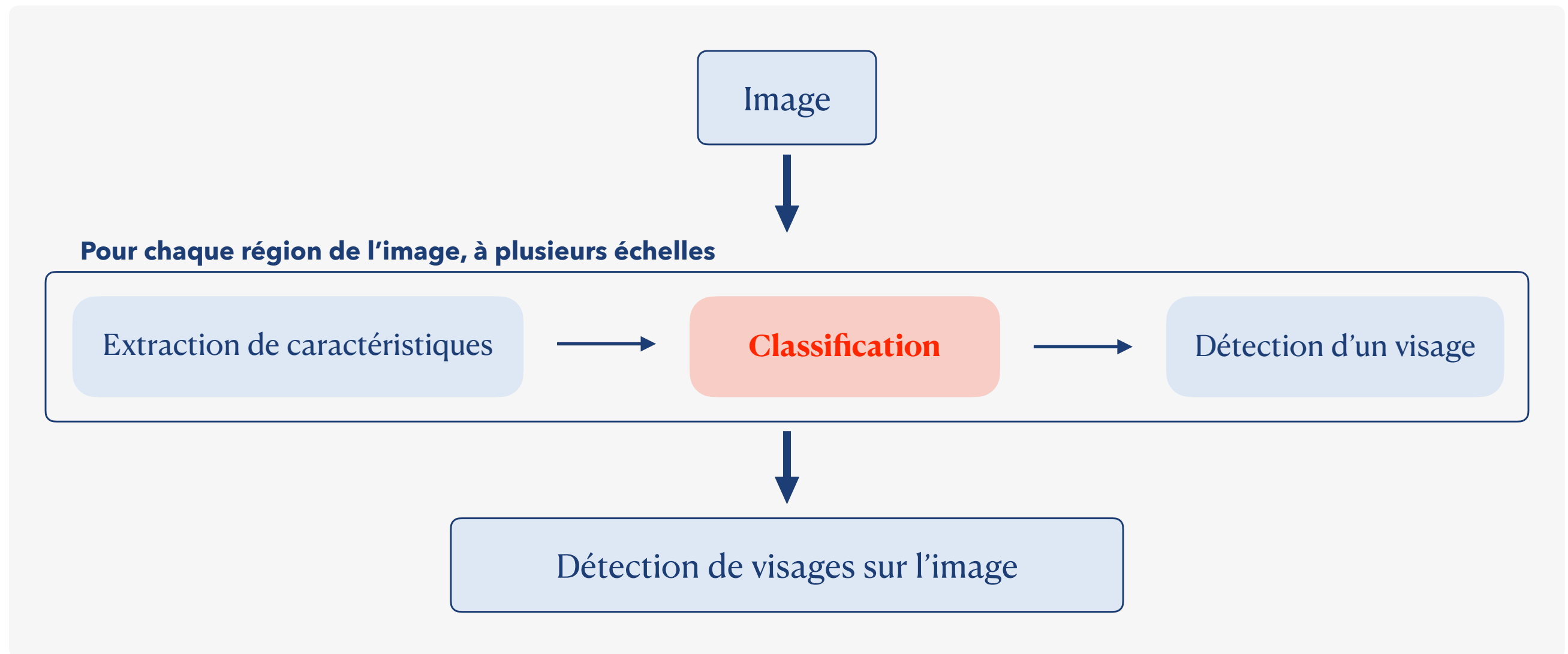


Figure: Processus de détection de visages avec un algorithme de type HOG+SVM

II. Classification

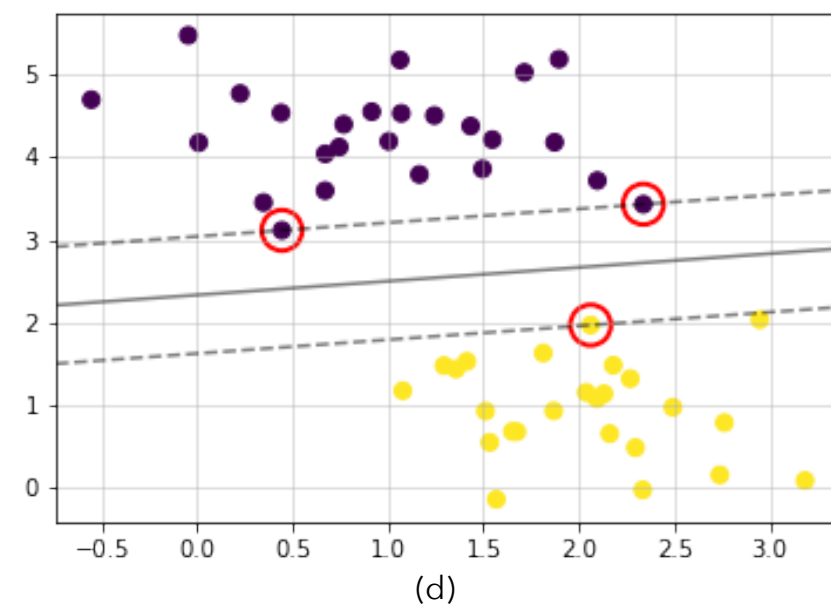
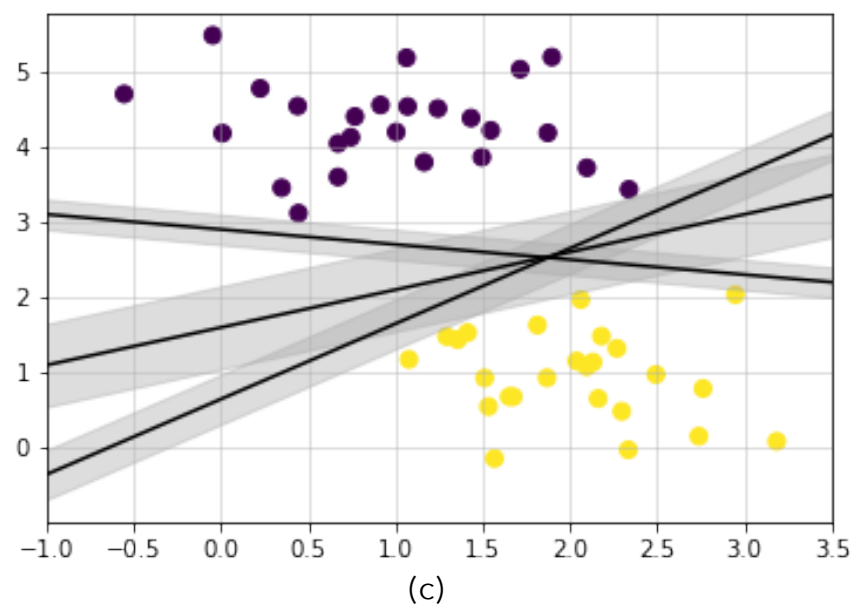
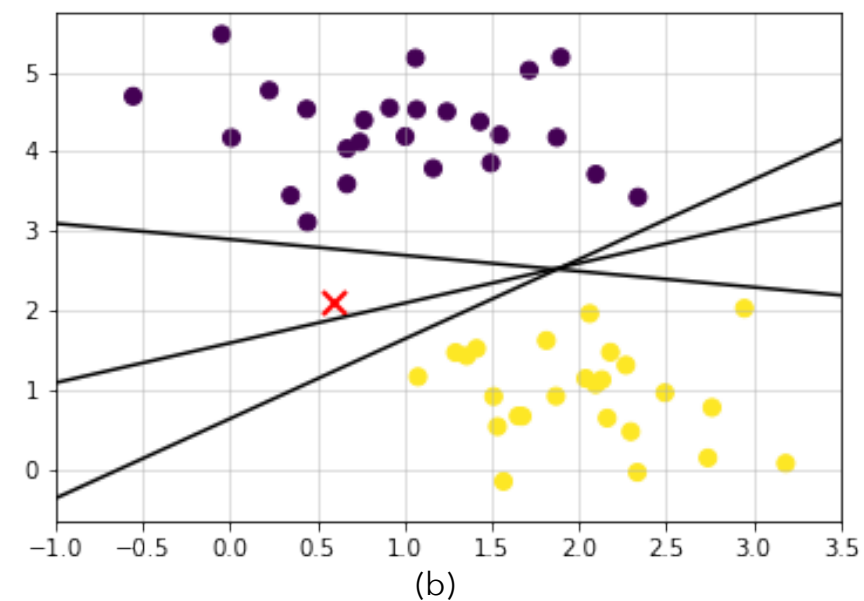
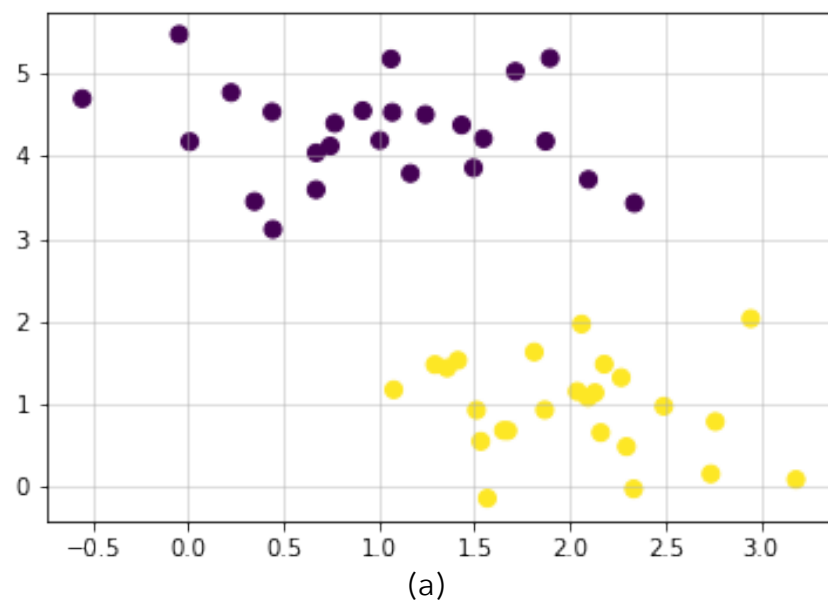
2) Machines à vecteurs de support (SVM)

- ▶ Théorie de VAPNIK-CHERVONENKIS, années 70
- ▶ **[2]** O. CHAPPELLE, P. HAFFNER ET V.N. VAPNIK « Support vector machines for histogram-based image classification » 1999

II. Classification

2) Machines à vecteurs de support (SVM)

- ▶ Théorie de VAPNIK-CHERVONENKIS, années 70
- ▶ [2] O. CHAPPELLE, P. HAFFNER ET V.N. VAPNIK « Support vector machines for histogram-based image classification » 1999



Figures: Recherche de l'hyperplan optimal en dimension 2

II. Classification

2) Machines à vecteurs de support (SVM)

Recherche d'un hyperplan séparateur

Soit $X \in (\mathbb{R}^n)^{\mathbb{N}}$, classe une application de X vers $\{-1, 1\}$. Trouver un hyperplan séparateur entre les classes -1 et 1 , c'est trouver un vecteur de poids w et un biais b tels que pour tout point x de X ,

$$\text{classe}(x) \times (w^{\top} \cdot x + b) \geq 0$$

Marge, vecteurs de support

Soit $x_k \in X = \{x_1, \dots, x_q\}$. En notant $\mathcal{H}$ l'hyperplan affine de vecteur de poids w et de biais b , on a :

$$d(x_k, \mathcal{H}) = \frac{\text{classe}(x_k) \times (w^{\top} \cdot x_k + b)}{\|w\|}$$

Alors, on définit la **marge** de $\mathcal{H}$ par rapport à l'ensemble de points X de la manière suivante:

$$\text{marge}(\mathcal{H}/X) = \min_k d(x_k, \mathcal{H}) = d(X, \mathcal{H})$$

Les x_k qui atteignent cette distance sont appelés **vecteurs de support**.

II. Classification

2) Machines à vecteurs de support (SVM)

Étant donné X , on cherche donc l'hyperplan affine $\mathcal{H}$ qui maximise la quantité $\text{marge}(\mathcal{H}/X)$.

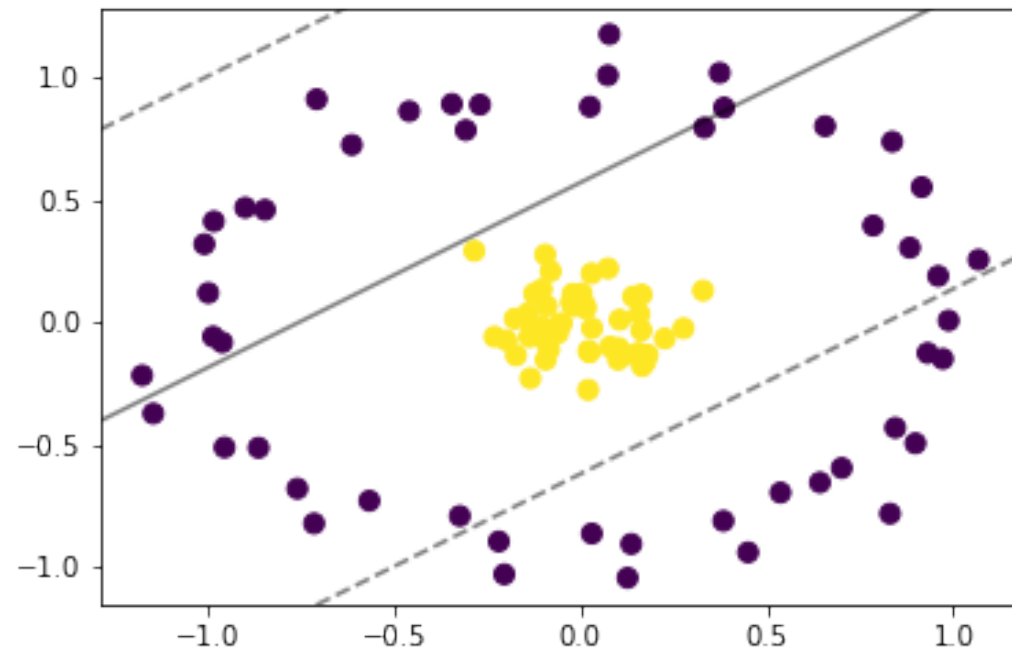
Formulation du problème d'optimisation de la marge

$$\begin{cases} \text{minimiser} & \frac{\|w\|^2}{2} \\ \text{sous les contraintes} & \forall k, \text{classe}(x_k) \times (w^\top \cdot x_k + b) \geq 1 \end{cases}$$

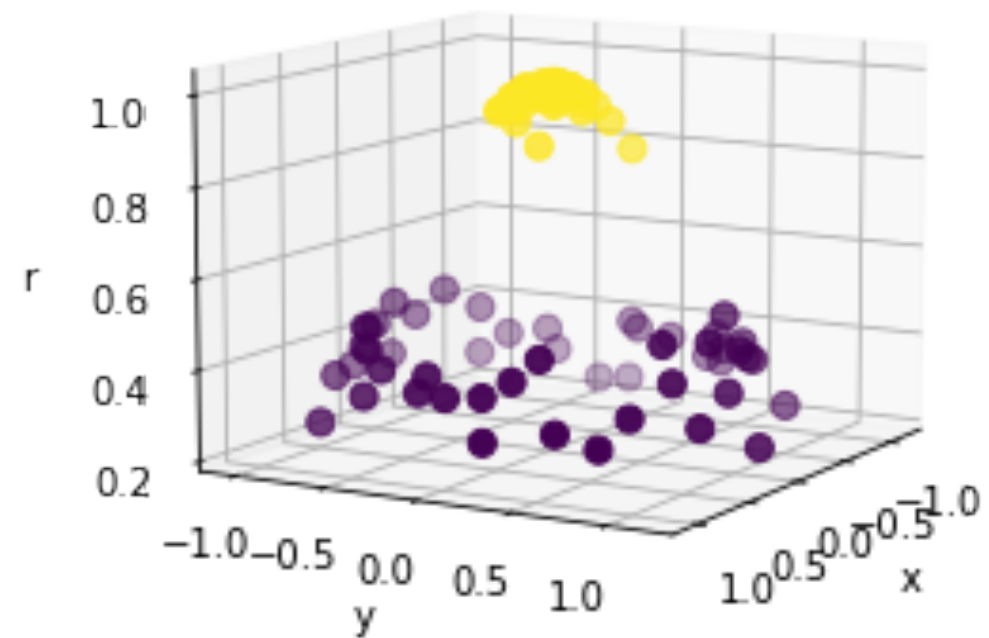
► Problème d'optimisation quadratique

II. Classification

2) Machines à vecteurs de support (SVM)



(a)



(b)

$$\begin{aligned} \Phi : \mathbb{R}^2 &\rightarrow \mathbb{R}^3 \\ (x, y) &\mapsto (x, y, r = \exp(-x^2 - y^2)) \end{aligned}$$

(c)

Figures: Illustration d'un cas où les données ne sont pas directement linéairement séparables.
(a) Avant redescription. (b) Après redescription. (c) Transformation.

III. Mise en place de l'algorithme

III. Mise en place de l'algorithme

1) Structure du projet, entraînement du modèle

- Utilisation de PYTHON
- Jeux de données : jeu d'entraînement (*training set*), jeu de test (*test set*)

HOG, SVM

► Scikit Learn

Fenêtre glissante (sliding window)



Figure 1: La fenêtre glissante parcourt les différentes régions de l'image

Pyramide d'image (image pyramid)

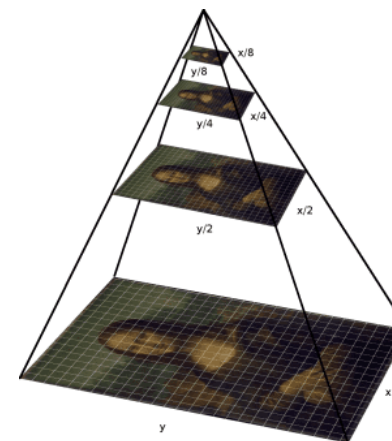


Figure 2: Une pyramide à 4 couches

Non-maximum suppression

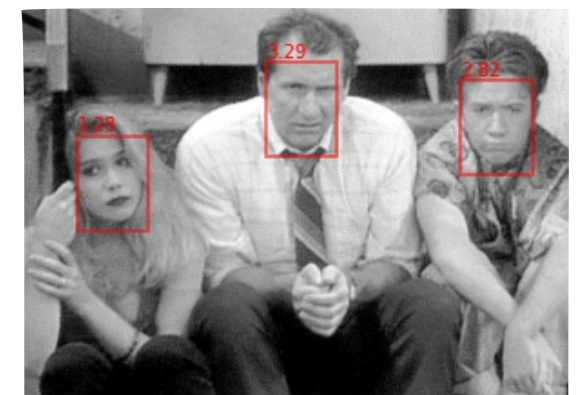
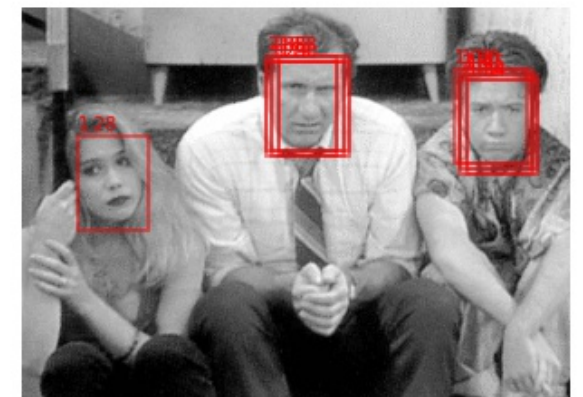


Figure 3: Avant/Après NMS

III. Mise en place de l'algorithme

2) Mesure des performances

Précision, rappel

$$\text{Précision} = \frac{\text{Nombre de visages correctement détectés}}{\text{Nombre de visages détectés}}$$

$$\text{Rappel} = \frac{\text{Nombre de visages correctement détectés}}{\text{Nombre de visages}}$$

Score F1

$$F_1 = 2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Courbes ROC

$$\text{Taux de vrais positifs} = \frac{\text{Vrais positifs}}{\text{Vrais positifs} + \text{Faux Négatifs}}$$

$$\text{Taux de faux positifs} = \frac{\text{Faux positifs}}{\text{Faux positifs} + \text{Vrais Négatifs}}$$

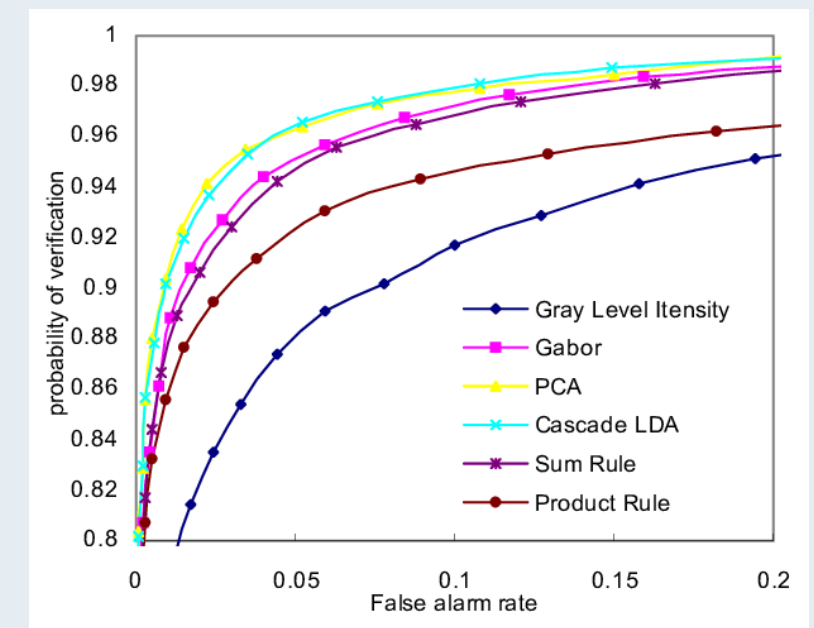


Figure: Illustration d'une courbe ROC.
Source: 10.1109/ICPR.2004.1334686. Wen Gao

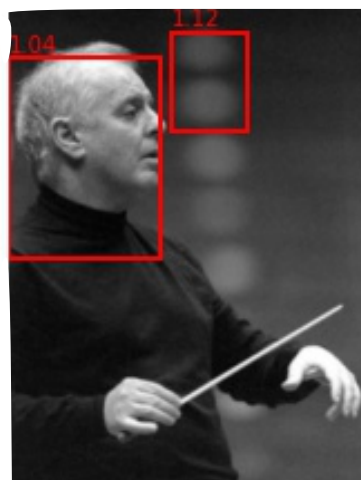
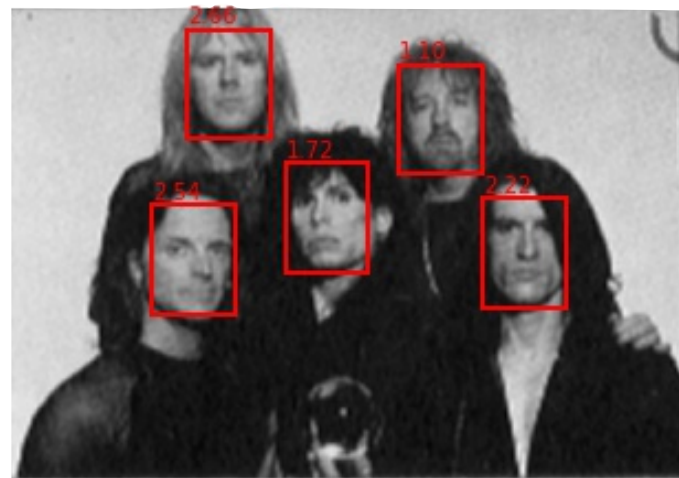
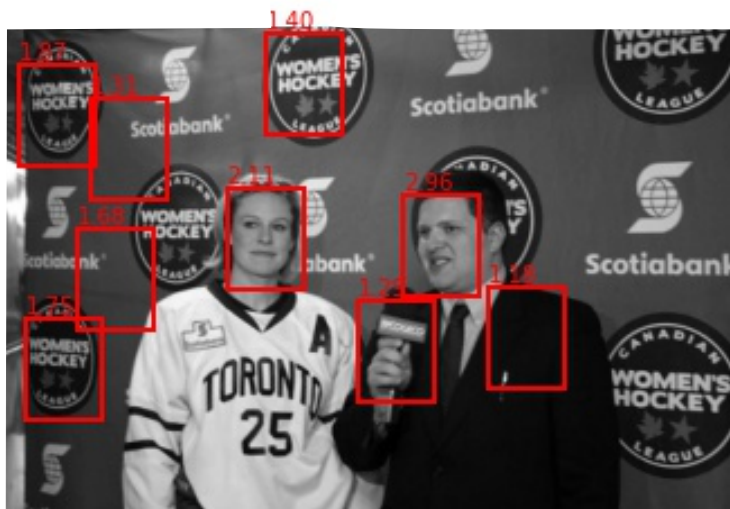
III. Mise en place de l'algorithme

2) Mesure des performances

Modèle étudié : MODEL_05, entraîné avec 42 000 images (dont 13 233 images contenant un visage, issues du dataset Labeled Faces in the Wild [5]) et amélioré par *hard negative mining*.

Jeu de test : 34 images, scènes variées

Exemples de détections



Résultats

Précision = 0.736

Rappel = 0.640

$$F_{1,SVM} = 0.684$$

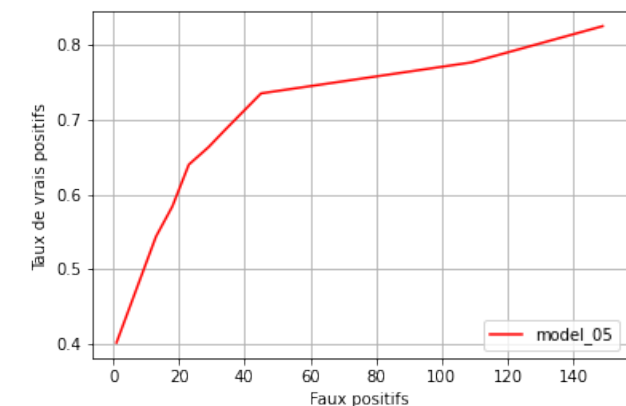
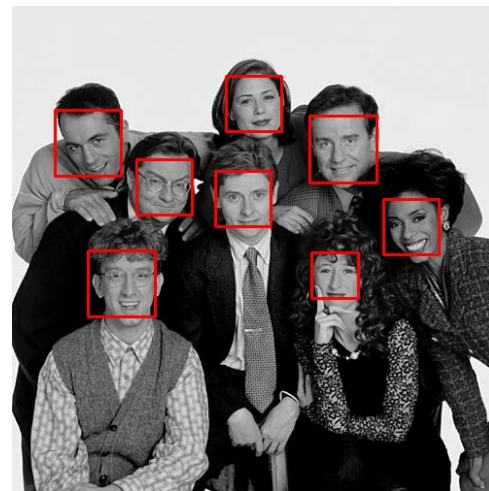


Figure: Courbe ROC du modèle étudié

Conclusion



(a)



(b)

Figures: (a) Utilisation de mon modèle. (b) Utilisation du modèle de détection de visages basé sur les CNN de la bibliothèque Dlib

$$F_{1,CNN} = 0.980 > F_{1,SVM} = 0.684$$

De par son faible coût en calculs, la faible taille de ses modèles ou encore sa robustesse aux variations d'éclairage, **l'utilisation des algorithmes de type HOG+SVM demeure pertinente dans de nombreuses situations.**

- système de vidéosurveillance urbaine



Annexe

- Fichier **main.py**, qui permet d'effectuer des prédictions sur une image (1/3)

```
### Importation des bilibliothèques
import time
import os
import imutils
import joblib

import matplotlib.pyplot as plt
import numpy as np
from skimage import io, color, transform, feature

from TIPEutils import sliding_window, non_max_suppression

### Paramètre global
patches_shape = (62, 47)

### Chargement du modèle
filename = 'models/model_04.sav'
model = joblib.load(filename)

### Chargement de l'image à étudier
img_name = ...
path_to_img = 'img/{}'.format(img_name)
img = io.imread(path_to_img)

### Prétraitement de l'image
img = color.rgb2gray(img)

(h_max, w_max) = (300, 600) # L'image ne doit pas dépasser une taille maximale
if img.shape[0] > h_max or img.shape[1] > w_max:
    s = min(h_max / img.shape[0], w_max / img.shape[1])
    img = transform.rescale(img, s)
```

Annexe

- Fichier **main.py**, qui permet d'effectuer des prédictions sur une image (2/3)

```
scale = 1.0
# Les patches auront une taille Ni*Nj
Ni, Nj = (int(scale * e) for e in patches_shape)

indices, patches, pred = [], [], []

start = time.time()

# On boucle à travers les étages de la pyramide d'image
for (l, resized) in enumerate(transform.pyramid_gaussian(img, downscale=2), start = 1):
    try:
        indices, patches = zip(*sliding_window(resized, patches_shape, scale=scale))
    except ValueError:
        break

indices, patches = zip(*sliding_window(resized, patches_shape, scale=scale))

indices = np.array(indices)

# Extraction des caractéristiques HOG pour chaque patch
patches_hog = np.array([hog(patch) for patch in patches])

# Prédiction
labels = model.predict(patches_hog)

conf_seuil = 1.0

for k in range(len(indices)):
    if labels[k] == 1 and model.decision_function(patches_hog)[k] > conf_seuil:
        (i, j) = indices[k]
        pred.append((j*1, i*1, (j + Nj)*1, (i + Ni)*1,
model.decision_function(patches_hog)[k]))
```

Annexe

- Fichier **main.py**, qui permet d'effectuer des prédictions sur une image (3/3)

```
pred = np.array(pred)

# Application de l'algorithme de NMS
nms_pred = non_max_suppression(pred, seuil=0.2)

### Affichage des résultats
fig, ax = plt.subplots()
ax.imshow(img, cmap='gray')
ax.axis('off')

for (y1, x1, y2, x2, conf) in nms_pred:
    rect = plt.Rectangle((y1, x1), y2-y1, x2-x1, edgecolor='red', alpha=0.5, lw=5,
facecolor='none')
    ax.add_patch(rect)
    ax.text(y1, x1-5, f'{conf:.2f}', color='red', weight='bold')

plt.show()

# Enregistrement du résultat
fig.savefig('output/{}'.format(img_name))

print("\nTemps d'exécution: {} secondes\n".format(round(time.time()-start, 2)))
```

Annexe

- Fichier **TIPUtils.py**, qui contient les fonctions de fenêtre glissante et de pyramide d'image (1/2)

```
def sliding_window(img, patch_size, istep=4, jstep=4, scale=1.0):  
  
    Ni, Nj = (int(scale * s) for s in patch_size)  
    # La sliding window aura une taille Ni*Nj  
  
    for i in range(0, img.shape[0] - Ni, istep):  
        for j in range(0, img.shape[1] - Nj, jstep):  
            patch = img[i:i + Ni, j:j + Nj]  
            if scale != 1:  
                patch = transform.resize(patch, patch_size)  
            yield (i, j), patch
```

Annexe

- Fichier **TIPEutils.py**, qui contient les fonctions de fenêtre glissante et de pyramide d'image (2/2)

```
def non_max_suppression(boxes, seuil=0.3):
    if len(boxes) == 0:
        return []

    l = []

    y1, x1, y2, x2 = boxes[:,0], boxes[:,1], boxes[:,2], boxes[:,3]

    # On calcule les aires des boxes
    aires = (x2 - x1 + 1) * (y2 - y1 + 1)    # le + 1 permet d'éviter la division par zéro

    indexes = np.argsort(boxes[:,4])

    while len(indexes) > 0:
        lst = len(indexes) - 1
        i = indexes[lst]
        l.append(i)

        # On cherche les intersections entre la box courante et toutes les autres
        y1_inter = np.maximum(y1[i], y1[indexes[:lst]])
        x1_inter = np.maximum(x1[i], x1[indexes[:lst]])
        y2_inter = np.minimum(y2[i], y2[indexes[:lst]])
        x2_inter = np.minimum(x2[i], x2[indexes[:lst]])

        h_inter = np.maximum(0, y2_inter - y1_inter + 1)
        w_inter = np.maximum(0, x2_inter - x1_inter + 1)
        aires_inter = (w_inter * h_inter)

        overlap = aires_inter / aires[indexes[:lst]]

        indexes = np.delete(indexes, np.concatenate(([lst], np.where(overlap > seuil)
[0])))

    return boxes[l]
```

Annexe

- Fichier **model_training.py**, qui permet d'entraîner un modèle (1/2)

```

### Importation des bibliothèques
...

faces = fetch_lfw_people()           # Importation des visages du dataset LFW
positive_patches = faces.images      # Ces images seront nos "positives patches"

# Pour les "negatives patches", on prend des images de thèmes variés qui ne contiennent pas de
visage
imgs_to_use = ['brick', 'text', 'moon', 'grass', 'gravel',
               'page', 'clock', 'immunohistochemistry',
               'chelsea', 'coffee', 'hubble_deep_field', 'cell']
images = [color.rgb2gray(getattr(data, name)())
          for name in imgs_to_use]

def extract_patches(img, N, scale=1.0, patch_size=positive_patches[0].shape):
    ''' Extraction des patches pour une image '''
    extracted_patch_size = tuple((scale * np.array(patch_size)).astype(int))
    extractor = PatchExtractor(patch_size=extracted_patch_size,
                              max_patches=N, random_state=0)
    patches = extractor.transform(img[np.newaxis])
    if scale != 1:
        patches = np.array([transform.resize(patch, patch_size)
                            for patch in patches])
    return patches

negatives_patches = np.vstack([extract_patches(im, 1000, scale)
                              for im in images for scale in [0.5, 1.0, 2.0]])

X_train = np.array([hog(img) for img in chain(positive_patches, negatives_patches)])
y_train = np.zeros(X_train.shape[0])
y_train[:positive_patches.shape[0]] = 1      # les positifs auront le label 1, les négatifs 0

```

Annexe

- Fichier **model_training.py**, qui permet d'entraîner un modèle (2/2)

```
### Entraînement du modèle
grid = GridSearchCV(LinearSVC(), {'C': [1.0, 2.0, 4.0, 8.0]})
grid.fit(X_train, y_train)
model = grid.best_estimator_

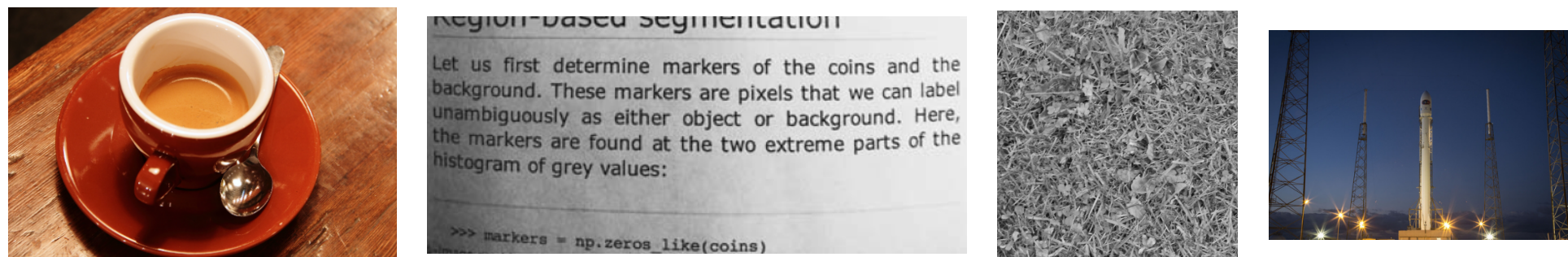
### Export du modèle
filename = '/models/model_04.sav'
joblib.dump(model, filename, compress=9)
```

Annexe

- Exemples d'images issues du dataset Labeled Faces in the Wild, utilisées pour l'entraînement des modèles (*positives patches*)



- Exemples d'images issues d'un dataset de la bibliothèque sklearn, utilisées pour l'entraînement des modèles (*negatives patches*)



- Exemples d'images issues du dataset WIDER, utilisées pour constituer le jeu de test

